ALARM CLOCK

A Project Report

Submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

CS-IT

Operating System and System Programming (25CS2104E)

By

PRIYAVALLIKA.M-2520090183

SHAMIKA HASINI-2520080033

SAMYUKTHA-2520090157

Under the Esteemed Guidance of

DR. M. RAGHUPATHI

PROFESSOR

Department of Computer Science and Engineering

KLH

2026-2027



DECLARATION

We hereby declare that the project report entitled “ALARM CLOCK” is a record of the bonafide work carried out by us as part of the Operating System and System Programming course. The work presented in this report was developed using C programming and Linux/POSIX system programming mechanisms and has been prepared for academic purposes.

Student 1: PRIYAVALLIKA .M (2520090183)

Student 2: SHAMIKA HASINI (2520080033)

Student 3: SAMYUKTHA (2520090157)

Date: 03/10/2026

KLH

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the project report entitled “ALARM CLOCK” is a bonafide work carried out by PRIYAVALLIKA .M , SHAMIKA HASINI, SAMYUKTHA ,in partial fulfillment of the requirements for the course Operating System and System Programming (25CS2104E) during the academic year 2026–27.

Guide: M.RAGHUPATHI

Course Coordinator: KOTTE ANUSHA

Head of the Department: [NAME]

Signature of the Guide Signature of the Course Coordinator Signature of the HOD

ACKNOWLEDGEMENT

We express our sincere gratitude to everyone who supported and guided us during the development of the Alarm Clock project. We are thankful to our course coordinator and project guide for their valuable guidance, technical suggestions, encouragement, and continuous support.

We also thank the Department of Computer Science and Engineering for providing the facilities and learning environment required to complete this project. We extend our thanks to the teaching and non-teaching staff, classmates, friends, and family members for their cooperation and encouragement.

Student 1: PRIYAVALLIKA

Student 2: SHAMIKA HASINI

Student 3: SAMYUKTHA

ABSTRACT

The Alarm Clock project is a Linux-based command-line application developed in C to schedule and trigger alarms at a specified time. The project demonstrates operating system and system programming concepts through Linux and POSIX mechanisms.

The application provides a simple menu through which a user can set an alarm, view an active alarm, cancel an alarm, view the current system time, and exit the application. When an alarm is set, the program creates a child process using fork(). The child process manages a POSIX timer and waits for the scheduled time. When the timer expires, a signal is generated and handled by the program, causing the alarm notification to be displayed.

The project also demonstrates signal handling using SIGUSR1 and SIGTERM, process synchronization using waitpid(), process control using kill(), and time management using functions such as time(), localtime(), mktime(), and strftime(). Input validation is included to prevent invalid alarm times.

Overall, the project provides a practical demonstration of process management, signal handling, POSIX timers, system calls, process synchronization, process termination, and Linux command-line programming.

INDEX

S. No.	Chapter	Topics / Subtopics
	Acknowledgement	i
	Abstract	ii
1	Introduction	9–10
	1.1 Background of the Project	
	1.2 Problem Statement	
	1.3 Scope of the Project	
	1.4 Objectives of the Project	
	1.5 Importance of the Application	
	1.6 Target Users / Audience	
2	OS Concepts and Background Study	11–12
	2.1 Operating System Concepts Related to the Project	
	2.2 Existing Methods / Algorithms	
	2.3 Proposed Approach	
	2.4 Advantages of the Proposed Approach	
3	System Requirements	13–14
	3.1 Hardware Requirements	
	3.2 Software Requirements	
	3.3 Operating System Environment – Ubuntu / WSL	
	3.4 Development Tools and Compilers	
4	Technology and Tools Used	15–16
	4.1 C Programming	
	4.2 GCC Compiler	
	4.3 Linux System Calls / APIs	
	4.4 POSIX Timers	
	4.5 Linux Signals	

	4.6 Linux Commands and Utilities	
5	System Architecture and Design	17–18
	5.1 Overall System Architecture	
	5.2 System Workflow	
	5.3 Flowchart / Data Flow	
	5.4 Module Description	
	5.5 Input and Output Design	
6	Implementation	19–20
	6.1 Module-wise Implementation	
	6.2 Algorithm / Pseudocode	
	6.3 Core C Implementation	
	6.4 Linux System Calls / APIs Used	
	6.5 Process Implementation	
	6.6 Signal and Timer Implementation	
	6.7 Synchronization and Termination	
	6.8 Input Validation	
7	Execution and Demonstration	21–24
	7.1 Compilation in Ubuntu	
	7.2 Execution Commands	
	7.3 Sample Inputs and Outputs	
	7.4 Screenshots of Execution	
	7.5 Demonstration of OS Concepts	
8	Conclusion	25
	8.1 Summary of the Project	
	8.2 Objectives Achieved	
	8.3 Results Achieved	
	8.4 Skills and	

	Learning Outcomes	
9	References	26
	9.2 Linux / GNU Documentation	
	9.3 Online Resources	

CHAPTER – 1

INTRODUCTION

1.1 Background of the Project

An alarm clock is a time-based application that monitors the system clock and provides a notification when a scheduled time is reached. In this project, the alarm clock is implemented as a Linux command-line application using C and POSIX APIs.

The project demonstrates how operating system facilities can be used in a practical application. The implementation uses a child process, POSIX timer, Linux signals, and process control functions.

1.2 Problem Statement

A simple alarm application needs to accept a user-defined time, wait efficiently until that time, notify the user, and provide a way to cancel the alarm. The project addresses these requirements while demonstrating core Linux system programming concepts.

1.3 Scope of the Project

- Set an alarm using HH:MM input.
- Display the currently scheduled alarm.
- Cancel an active alarm.
- Display the current system time.
- Trigger an alarm notification at the scheduled time.
- Use Linux processes and POSIX timers for alarm management.
- Validate user input and handle invalid time formats.

1.4 Objectives of the Project

Develop a functional Linux-based alarm clock using C.
Apply process creation and management using fork().
Implement alarm scheduling using POSIX timers.
Use Linux signals for alarm triggering and cancellation.
Use process synchronization through waitpid().
Practice Linux system calls and POSIX time-related APIs.
Provide a simple command-line interface.

1.5 Importance of the Application

The application connects theoretical operating system concepts with observable real-world behavior: scheduling a task for a future time and handling the event using processes, timers, and signals.

1.6 Target Users / Audience

The application is intended primarily for academic demonstration and learning in a Linux/system-programming environment. It can also be used as a simple command-line alarm utility.

CHAPTER – 2

OS CONCEPTS AND BACKGROUND STUDY

2.1 Operating System Concepts Related to the Project

Concept	Application in the Project
Process Management	fork() creates a child process dedicated to alarm handling.
Parent and Child Processes	The parent manages the menu while the child waits for and triggers the alarm.
Signals	SIGUSR1 is used for the timer-triggered event and SIGTERM for cancellation.
Signal Handling	sigaction() associates signal handlers with the required signals.
POSIX Timers	timer_create() and timer_settime() provide scheduled timer functionality.
System Calls / APIs	fork(), kill(), waitpid(), pause(), and time-related APIs connect the application to OS services.
Process Synchronization	waitpid() allows the parent to wait for the alarm child to finish.
Process Termination	The alarm process exits after completion or cancellation.
Efficient Waiting	pause() suspends the process until a signal arrives.
Time Management	time(), localtime(), mktime(), and strftime() manage and display time.

2.2 Existing Methods / Algorithms

A simple alarm can repeatedly check the current time in a loop. Such polling can repeatedly consume CPU time. This project instead uses an operating-system timer and a dedicated child process.

2.3 Proposed Approach

The parent process manages the command-line menu. After the user sets an alarm, fork() creates a child process. The child configures a POSIX timer using the absolute scheduled time. When the timer expires, SIGUSR1 is delivered to the child and the alarm is displayed. If the user cancels the alarm, the parent sends SIGTERM to the child.

2.4 Advantages of the Proposed Approach

- Clear separation between menu handling and alarm handling.

- Efficient waiting using pause() rather than continuous time polling.
- Direct demonstration of Linux process and signal mechanisms.
- Uses standard C/POSIX interfaces available in Linux.
- Provides cancellation and input validation.
- Easy to demonstrate and test in Ubuntu/WSL.

CHAPTER – 3

SYSTEM REQUIREMENTS

3.1 Hardware Requirements

- Computer or laptop.
- Minimum 4 GB RAM for the project environment.
- Processor capable of running Ubuntu/Linux or WSL.
- Speaker/audio output if terminal bell notification is enabled.

3.2 Software Requirements

- Ubuntu / Linux environment.
- Windows Subsystem for Linux (WSL), if using Ubuntu on Windows.
- GCC compiler.
- C programming environment.
- VS Code / Cursor / Vim or another text editor.
- Git and GitHub for version control and sharing.

3.3 Operating System Environment – Ubuntu / WSL

The project was developed and tested in Ubuntu running through WSL on Windows. Linux provides the process, signal, timer, and time-related interfaces required by the implementation.

```
priylvn@LAPTOP-NOJKE600:~$ cd ~/alarm_clock
ls
README.md  alarm_clock  run_alarm.sh  screenshots  src
priylvn@LAPTOP-NOJKE600:~/alarm_clock$ find . -maxdepth 2 -type
f
./.git/index
./.git/HEAD
./.git/config
./.git/description
./.git/COMMIT_EDITMSG
./alarm_clock
./README.md
./run_alarm.sh
./src/alarm_clock.c
./.gitignore
priylvn@LAPTOP-NOJKE600:~/alarm_clock$
```

3.4 Development Tools and Compilers

The C source file is compiled using GCC. The project is organized with a source directory, README documentation, launcher script, and Git configuration.

CHAPTER – 4

TECHNOLOGY AND TOOLS USED

Technology / Tool	Purpose
C Programming	Implementation of alarm logic, menu, processes, signals, and time functions.
GCC Compiler	Compiles src/alarm_clock.c into the executable.
Linux System Calls / APIs	fork(), kill(), waitpid(), pause(), and related POSIX interfaces.
POSIX Timers	timer_create() and timer_settime() schedule the alarm.
Linux Signals	SIGUSR1 and SIGTERM communicate alarm trigger and cancellation.
Time APIs	time(), localtime(), mktime(), and strftime() manage and display time.
Shell Script	run_alarm.sh compiles and launches the application in a separate WSL terminal.
Git / GitHub	Version control, project storage, and cloning/testing.

4.1 C Programming

C provides direct access to process-control and POSIX APIs and is suitable for demonstrating operating-system concepts.

4.2 GCC Compiler

GCC compiles the C source file and produces the alarm_clock executable.

4.3 Linux System Calls / APIs

Linux/POSIX functions provide process creation, signal delivery, process waiting, sleeping, and time management.

4.4 POSIX Timers

The timer is created with timer_create() and scheduled with timer_settime() using TIMER_ABSTIME.

4.5 Linux Signals

SIGUSR1 is associated with the timer event and SIGTERM is used for cancellation.

4.6 Linux Commands and Utilities

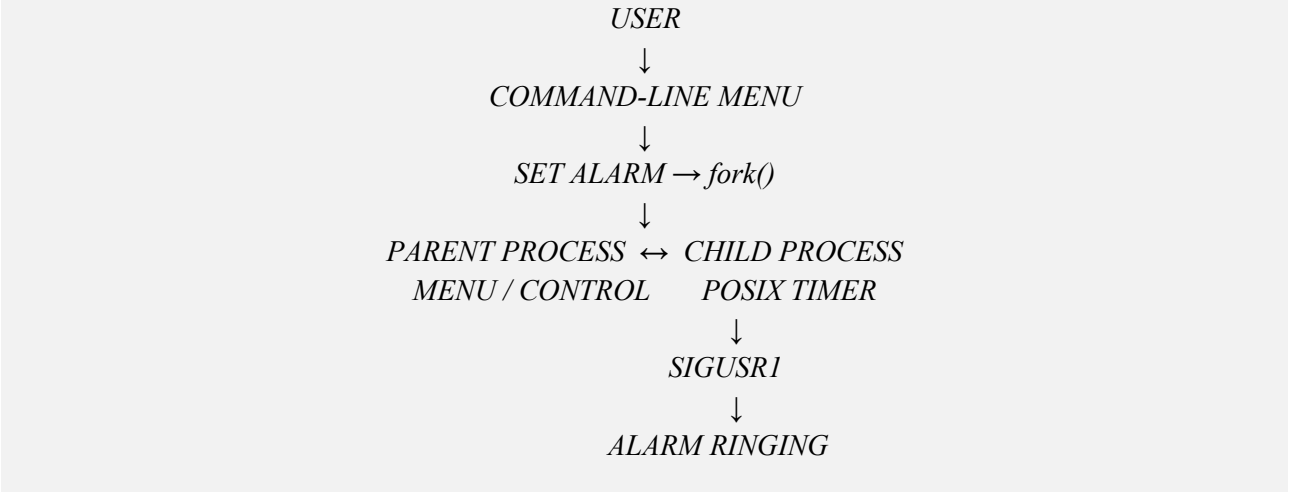
gcc compiles; cd changes directory; chmod enables the launcher; ./run_alarm.sh starts the project; git manages the repository.

CHAPTER – 5

SYSTEM ARCHITECTURE AND DESIGN

5.1 Overall System Architecture

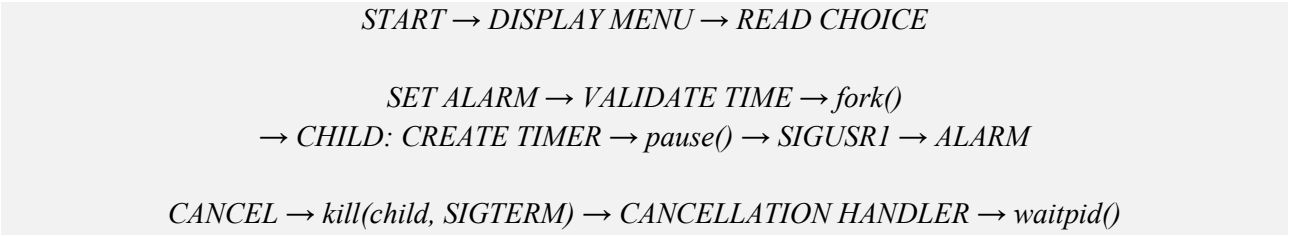
The application follows a parent-child process architecture.



5.2 System Workflow

1. Start the application.
2. Display the menu.
3. Read the user's choice.
4. For Set Alarm, read and validate HH:MM input.
5. Convert the requested time into a time_t value.
6. Create a child process using fork().
7. Configure a POSIX timer in the child.
8. Wait for SIGUSR1 or SIGTERM.
9. Display the alarm notification when the timer expires.
10. Allow the parent to cancel the alarm using SIGTERM.
11. Use waitpid() to synchronize with the child.

5.3 Flowchart / Data Flow



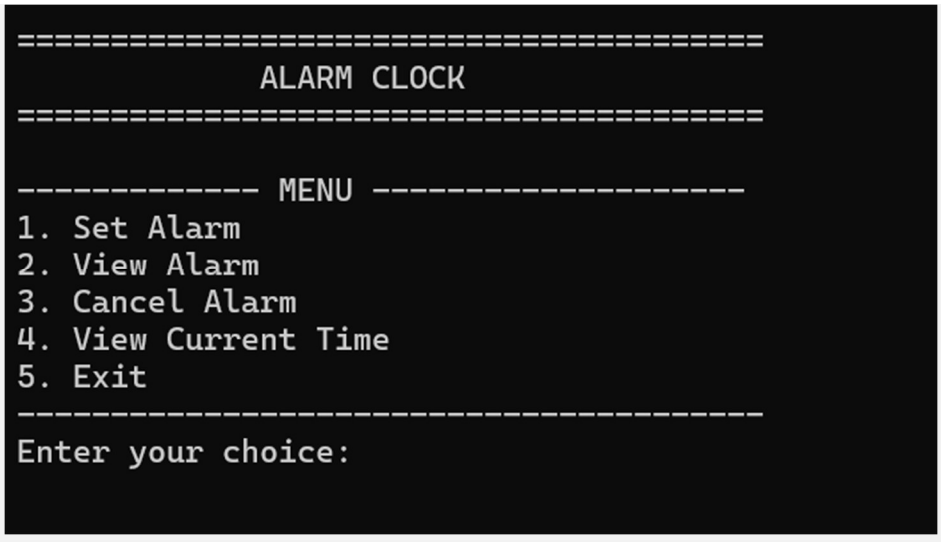
VIEW / CURRENT TIME → DISPLAY INFORMATION
EXIT → TERMINATE ACTIVE CHILD IF NEEDED → END

5.4 Module Description

Module	Description
Menu Module	Reads user choices and controls application flow.
Time Conversion Module	Validates HH:MM and converts it into scheduled system time.
Alarm Process Module	Runs in the child and manages the POSIX timer.
Signal Module	Handles SIGUSR1 for alarm trigger and SIGTERM for cancellation.
Display Module	Shows current time, scheduled alarm, status, and notifications.
Process Control Module	Uses fork(), kill(), waitpid(), and process IDs.

5.5 Input and Output Design

Input consists of a menu choice and alarm time in HH:MM format. Output includes current time, scheduled alarm, status, cancellation messages, and the alarm notification.



CHAPTER – 6

IMPLEMENTATION

6.1 Module-wise Implementation

The implementation is contained primarily in src/alarm_clock.c. It is divided logically into signal handlers, alarm-process logic, time conversion, display functions, and the main menu/process-control loop.

6.2 Algorithm / Pseudocode

1. Initialize alarm state and process ID.
2. Display the menu repeatedly.
3. If Set Alarm is selected, validate the entered time.
4. Create a child process using fork().
5. In the child, create and arm a POSIX timer.
6. Wait using pause() until a signal is received.
7. If SIGUSR1 is received, display the alarm notification.
8. If SIGTERM is received, cancel the alarm.
9. In the parent, continue the menu and support View/Cancel/Time operations.
10. On cancellation or exit, wait for the child process to terminate.

6.3 Core C Implementation

- sigaction() installs signal handlers.
- timer_create() creates the POSIX timer.
- timer_settime() schedules the absolute alarm time.
- fork() creates the alarm child process.
- kill() sends SIGTERM for cancellation.
- waitpid() synchronizes parent and child.
- time(), localtime(), mktime(), and strftime() manage time.

6.4 Linux System Calls / APIs Used

API / Function	Purpose
fork()	Creates the child alarm process.
kill()	Sends SIGTERM to the child during cancellation.
waitpid()	Waits for the child process to terminate.
pause()	Suspends the child until a signal arrives.
sigaction()	Registers signal handlers.
timer_create()	Creates a POSIX timer.

timer_settime()	Schedules the timer.
timer_delete()	Releases the timer after completion.
time()	Obtains current system time.
localtime()	Converts time to local calendar representation.
mktime()	Converts calendar representation to time_t.
strftime()	Formats date/time for display.

6.5 Process Implementation

fork() creates the alarm child. The parent retains the child's PID so it can view the alarm state and send a cancellation signal.

6.6 Signal and Timer Implementation

The child configures SIGUSR1 as the timer notification signal and SIGTERM for cancellation. The POSIX timer uses CLOCK_REALTIME and TIMER_ABSTIME for a scheduled clock time.

6.7 Synchronization and Termination

The parent uses waitpid() after cancellation and before final termination to wait for the child process. The child exits after the alarm completes or is cancelled.

6.8 Input Validation

The program checks that the input can be parsed as hours and minutes and that hours are within 0–23 and minutes are within 0–59. Invalid inputs are rejected with a clear message.

CHAPTER – 7

EXECUTION AND DEMONSTRATION

7.1 Compilation in Ubuntu

```
cd ~/alarm_clock
gcc src/alarm_clock.c -o alarm_clock
```

The provided launcher can also compile automatically before starting the program.

7.2 Execution Commands

```
cd ~/alarm_clock
./run_alarm.sh
```

The launcher is configured for the tested Ubuntu/WSL environment and starts the Alarm Clock in a separate terminal window.

7.3 Sample Inputs and Outputs

Test Case	Input	Expected / Observed Result
Set valid alarm	1 → HH:MM	Alarm successfully set; PID and scheduled time displayed.
View alarm	2	Scheduled alarm and ACTIVE status displayed.
Cancel alarm	3	Alarm process cancelled; cancellation confirmed.
Current time	4	Current system time displayed.
Invalid time	1 → 25:70	Invalid time format message displayed.
Invalid text	1 → hello	Invalid time format message displayed.
Duplicate alarm	1 while active	User is asked to cancel the active alarm first.
Exit	5	Alarm Clock closed after active child is handled.

7.4 Screenshots of Execution

```
========================================
             ALARM CLOCK
========================================

------------ MENU --------------------
1. Set Alarm
2. View Alarm
3. Cancel Alarm
4. View Current Time
5. Exit
----------------------------------------
Enter your choice:
```

```
========================================
             ALARM CLOCK
========================================

------------ MENU --------------------
1. Set Alarm
2. View Alarm
3. Cancel Alarm
4. View Current Time
5. Exit
----------------------------------------
Enter your choice: 1

Enter alarm time (HH:MM): 17:55

Alarm successfully set.
Alarm Process ID: 702
Alarm Time: 2026-10-03 17:55:00
```

```
========================================
            ALARM RINGING!
========================================

Alarm completed.
```

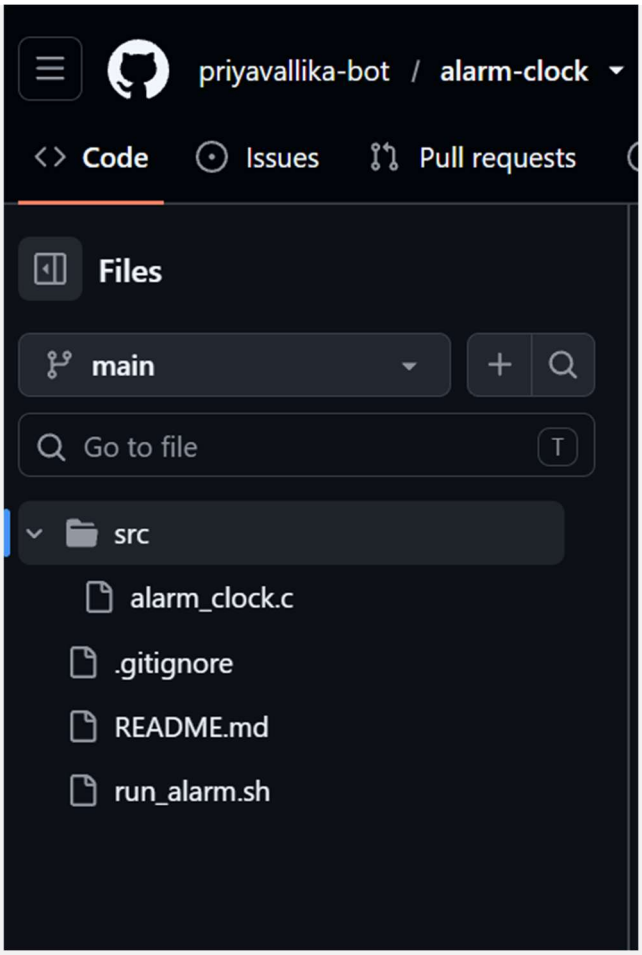
```
------------- MENU --------------------
1. Set Alarm
2. View Alarm
3. Cancel Alarm
4. View Current Time
5. Exit
---------------------------------------
Enter your choice: 2

Scheduled Alarm: 2026-10-03 17:59:00
Status: ACTIVE
```

```
------------- MENU --------------------
1. Set Alarm
2. View Alarm
3. Cancel Alarm
4. View Current Time
5. Exit
---------------------------------------
Enter your choice: 3

Alarm process cancelled.

Alarm cancelled successfully.
```



7.5 Demonstration of OS Concepts

- Process creation: fork() creates a separate alarm child process.
- Signal handling: SIGUSR1 triggers the alarm handler; SIGTERM triggers cancellation.
- Timer management: POSIX timer schedules the alarm.
- System calls/APIs: Linux/POSIX functions provide OS services.
- Synchronization: waitpid() coordinates parent and child termination.
- Efficient waiting: pause() suspends the child until a signal is delivered.
- Time management: time-related functions convert and format scheduled/current times.

CHAPTER – 8

CONCLUSION

8.1 Summary of the Project

The Alarm Clock project implements a Linux command-line alarm application using C and POSIX system programming mechanisms. It allows users to set, view, cancel, and monitor alarms while demonstrating the interaction between application code and operating system services.

8.2 Objectives Achieved

- Functional Linux-based alarm clock implemented.
- Alarm management separated into a child process.
- POSIX timer functionality implemented.
- Signals used for alarm triggering and cancellation.
- Parent-child synchronization implemented.
- Input validation and user-friendly CLI messages provided.
- Project packaged with documentation and uploaded to GitHub.

8.3 Results Achieved

The project was compiled and tested successfully in Ubuntu/WSL, and the completed project was uploaded to GitHub for version control and sharing.

8.4 Skills and Learning Outcomes

- Linux process creation and process IDs.
- Signal generation and handling.
- POSIX timers for time-based scheduling.
- Linux/POSIX APIs from C.
- Parent-child process coordination.
- Compiling and running C programs in Ubuntu.
- Git and GitHub for version control.
- Technical documentation and demonstration.

CHAPTER – 9

REFERENCES

9.1 Linux / GNU Documentation

- Linux manual pages for fork, kill, waitpid, pause, sigaction, timer_create, timer_settime, and related APIs.
- GNU GCC documentation.

9.2 Online Resources

- Official Linux / POSIX API documentation.
- Official Git and GitHub documentation.
- Course notes and laboratory material.

APPENDIX A – PROJECT STRUCTURE

```
alarm_clock/
├── src/
│   └── alarm_clock.c
├── screenshots/
├── run_alarm.sh
├── README.md
└── .gitignore
```